

VOLUME 4

COMPLETE VOLUME — CHAPTERS 4.1–4.10

Backend Architecture

Every Server-Side Decision Volume 0 and Volume 1 Deferred, Now Resolved

Document Title	Volume 4 — Backend Architecture (Combined)
Volume Reference	Volume 4
Chapters Included	4.1 through 4.10 (10 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 1–3

Table of Contents

This volume finalizes the backend: compute platform, database engine, schema, metadata storage, API contract, authentication, authorization, cloud topology, and S3 storage design — resolving every backend item Volume 0 and Volume 1 explicitly deferred.

Chapter	Title
4.1	Backend Overview
4.2	Database Design
4.3	Entity Relationship Model
4.4	Data Dictionary
4.5	Metadata Specification
4.6	API Specification
4.7	Authentication Design
4.8	Authorization Design
4.9	Cloud Architecture
4.10	Storage Architecture

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.1

Backend Overview

Finalizing What Volume 0 and Volume 3 Left Open

Document Title	Backend Overview
Chapter Reference	4.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Resolves	Volume 0 Ch.0.2 ("compute/auth provider finalized in Volume 4"); Volume 1 Ch.1.1 open questions.

1. Purpose

Volume 0 fixed AWS S3 for chunk storage and named Firebase as a likely auth/push candidate, but explicitly deferred the compute platform, API language/framework, and database engine to this volume. This chapter makes those three calls, each backed by an ADR since each has more than one reasonable option (Volume 0, Chapter 0.2's ADR rule).

2. ADR-007 — Compute Platform

ADR-007 — Backend Compute Platform

Status: Accepted

Context:

Collector upload traffic is bursty (field crews recording in shifts, not constant load) — NFR-SCL-02 requires scaling to a growing fleet without paying for idle capacity between bursts.

Options Considered:

- Always-on server (EC2/ECS Fargate) — predictable, but pays for idle capacity during low-usage hours.
- AWS Lambda + API Gateway — true pay-per-request, scales to zero, and colocates naturally with S3 for presigned-URL generation.

Decision:

Chosen: AWS Lambda (Node.js/TypeScript) behind API Gateway, one function per resource domain (auth-verify, projects, tasks, sessions, chunks, metadata).

Consequences:

- Cost tracks actual Collector activity rather than a fixed server bill — important for an MVP with an unproven usage curve.
- Cold-start latency is an accepted tradeoff, mitigated later (Volume 7) with provisioned concurrency only if measured latency requires it.

3. ADR-008 — Database Engine

ADR-008 — Database Engine

Status: Accepted

Context:

Projects, Tasks, Sessions, Chunks, and Metadata are deeply relational (a chunk belongs to a session belongs to a task belongs to a project; Admin dashboards join across all of them) and metadata integrity rules (BR-21/22) benefit from real constraints, not just application-level checks.

Options Considered:

- DynamoDB — scales effortlessly but models relational joins poorly; Admin's cross-Project reporting (FR-ADM-05/06) would need denormalized duplication.
- Firestore — pairs naturally with Firebase Auth, but same relational-modeling weakness as DynamoDB for this data shape.
- Amazon Aurora Serverless v2 (PostgreSQL) — full relational/ACID semantics, scales capacity up/down automatically with load, colocated in AWS alongside S3 and Lambda.

Decision:**Chosen:** Amazon Aurora Serverless v2 (PostgreSQL).**Consequences:**

- Chapter 4.2–4.4 can use a normal 3NF relational schema with foreign keys and CHECK constraints enforcing BR-18–23 at the database level, not just in application code.
- Serverless scaling matches the same bursty-load reasoning as ADR-007, keeping cost proportional to actual use.

4. ADR-009 — API Style

ADR-009 — API Style**Status:** Accepted**Context:**

Volume 0's Constitution already assumed a specific shape in its naming-convention example — "API endpoints: kebab-case, resource-oriented, e.g. `/projects/{id}/tasks`" — which is a REST convention, not a GraphQL one.

Options Considered:

- GraphQL — flexible querying, but adds a schema/resolver layer this app's fairly fixed set of screens (Chapter 2.5) doesn't need.
- REST, resource-oriented, versioned (`/v1/...`) — matches the Constitution's own naming example directly.

Decision:**Chosen:** REST, versioned, resource-oriented, detailed in full in Chapter 4.6.**Consequences:**

- No new query language/tooling to learn or secure; endpoint shapes map directly onto the Screen Inventory's data needs (Chapter 2.5).

5. Finalized Stack Summary

Layer	Choice
Compute	AWS Lambda (Node.js/TypeScript) + API Gateway
Database	Amazon Aurora Serverless v2 (PostgreSQL)
Object Storage	AWS S3 (fixed, Volume 0)

Layer	Choice
Authentication	Firebase Authentication (confirmed from Volume 0's leading candidate)
Push Notifications	Firebase Cloud Messaging (confirmed)
API Style	REST, versioned, resource-oriented
Monitoring/Logging	Amazon CloudWatch (Lambda + RDS metrics/logs)

6. What the Rest of Volume 4 Covers

- 4.2–4.4: the Aurora schema, its entity-relationship model, and full column-level data dictionary.
- 4.5: the formal metadata schema resolving FR-META-01–15.
- 4.6: the REST endpoint catalog.
- 4.7–4.8: how Firebase-issued tokens are verified and how roles/scoping (BR-18–20) are enforced server-side.
- 4.9–4.10: the AWS account/network topology and the S3 bucket/lifecycle design.

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.2

Database Design

The Aurora PostgreSQL Schema, Shaped by Volume 1's Roles and Rules

Document Title	Database Design
Chapter Reference	4.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Engine	Amazon Aurora Serverless v2 (PostgreSQL) — ADR-008, Chapter 4.1

1. Design Principles

- Normalized to 3NF for core entities — Projects, Tasks, Sessions, Chunks are never duplicated data, only referenced by foreign key, so an Admin's edit (FR-ADM-02) is instantly visible everywhere without a sync step.
- Metadata lives in its own table, not embedded in the chunks table, so its immutability rule (BR-22) can be enforced by a dedicated trigger without constraining the chunks table's own lifecycle fields (upload status, retry count).
- Every table name is plural snake_case; every column is snake_case — per Volume 0's Constitution naming table.
- Soft-delete (a deleted_at timestamp) is used wherever Volume 1 implies data must outlive its "active" state — most importantly chunk_metadata, which must remain queryable after local chunk cleanup (FR-META-14, BR-23) and after a Project is archived.

2. Core Tables

Table	Purpose
users	One row per Admin or Collector; role and org scoping (BR-20) live here.
projects	Admin-created Projects (FR-ADM-01).
tasks	Tasks within a Project (FR-ADM-02).
task_assignments	Many-to-many: which Collectors are assigned to which Task (FR-ADM-03/04).
sessions	One recording session — one Collector, one Task, a start time, and a Complete/In-Progress status (FR-SES-02).
chunks	One row per 10-minute chunk — sequence index, upload status, S3 object key, checksum.
chunk_metadata	One row per chunk — every FR-META-01–07 field, immutable except notes/tags (BR-22).
audit_log	Append-only record of Admin actions on Projects/Tasks/Assignments — supports future QA/rework workflow (Volume 1, Phase 3) without a schema change later.

3. Key Constraints Enforcing Volume 1's Business Rules

BR-18/BR-19 (only Admin creates/edits; a Collector never sees unassigned work): enforced at the application layer (Chapter 4.8, Authorization) — the database itself has no per-row visibility concept, so every query from a Collector's token is scoped by a WHERE task_assignments.user_id = :current_user clause the API layer always injects, never left optional.

BR-20 (Admin scoped to their own org, not cross-tenant): every Project row carries an org_id foreign key; every Admin-scoped query is filtered by the Admin's own org_id, identically enforced at the API layer.

BR-21 (a chunk isn't Complete until metadata is confirmed): chunks.status can only transition to 'complete' via a stored procedure that first checks a matching, non-null chunk_metadata row exists — not left to application code discipline alone.

BR-22 (system-generated metadata is immutable): a BEFORE UPDATE trigger on chunk_metadata rejects any change to a system-generated column, allowing only the notes/tags column to change.

BR-8/BR-11 (no chunk deleted before S3 confirms; no duplicate on retry): chunks.s3_object_key is UNIQUE, and chunks.local_deleted_at can only be set once chunks.status = 'complete' (Chapter 4.1's stored-procedure gate).

4. What This Chapter Defers

- Full column-level types/constraints — Chapter 4.3 (ERD) and Chapter 4.4 (Data Dictionary).
- The exact metadata field list and JSON shape for API payloads — Chapter 4.5.

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.3

Entity Relationship Model

How the Nine Core Tables Connect

Document Title	Entity Relationship Model
Chapter Reference	4.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Entity Relationship Diagram (Textual)

```

orgs —< users —< task_assignments >— tasks —< sessions —< chunks —1:1—
chunk_metadata
    |
    |< projects —< tasks (above)          | belongs to projects (via tasks)

users —< audit_log  (an Admin's action on a project/task is logged, never edited)

Legend: —< = one-to-many      —1:1— = one-to-one

```

2. Relationship Table

Relationship	Cardinality	Rule It Enforces
orgs → users	1:N	BR-20 — an Admin belongs to exactly one org; scopes every query.
orgs → projects	1:N	A Project belongs to exactly one org; Admins never see another org's Projects.
projects → tasks	1:N	FR-ADM-02 — Tasks exist only within a Project.
users ↔ tasks (via task_assignments)	M:N	FR-ADM-03/04 — a Collector can be assigned to many Tasks; a Task can have many Collectors.
tasks → sessions	1:N	A Session is always tied to exactly one Task and one Collector.
sessions → chunks	1:N	FR-CHK / Ch.1.8 UC-01 — a session accumulates one row per finalized chunk.
chunks → chunk_metadata	1:1	FR-META-08 — every chunk has exactly one, immutable metadata record.
users → audit_log	1:N	Append-only trail of every Admin write action.

3. Why Metadata Is 1:1, Not Embedded

Chapter 4.2 already explained the rationale (BR-22 immutability isolated from chunk lifecycle fields); the ERD makes the consequence explicit: `chunk_metadata` can never exist without a parent chunk (foreign key, NOT NULL, ON DELETE RESTRICT — a chunk row is never hard-deleted while its metadata must outlive it per BR-23, so cleanup uses the soft-delete pattern from Chapter 4.2 instead of a real DELETE).

4. What This Chapter Defers

- Column-level detail for every table shown here — Chapter 4.4 (Data Dictionary).
- The specific fields inside `chunk_metadata` — Chapter 4.5 (Metadata Specification).

Data Dictionary

Column-Level Definitions for Every Core Table

Document Title	Data Dictionary
Chapter Reference	4.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. users

Column	Type	Null?	Notes
id	uuid	No	Primary key.
org_id	uuid	No	FK → orgs.id (BR-20 scoping).
email	text	No	Unique; matches Firebase Auth account.
firebase_uid	text	No	Unique; links to Firebase Authentication (Chapter 4.7).
role	text	No	'admin' or 'collector' (Chapter 1.6); CHECK constraint restricts values.
display_name	text	Yes	
created_at	timestampz	No	

2. projects

Column	Type	Null?	Notes
id	uuid	No	Primary key.
org_id	uuid	No	FK → orgs.id.
name	text	No	FR-ADM-01.
description	text	Yes	
created_by	uuid	No	FK → users.id (must be role='admin', enforced at API layer, Chapter 4.8).
created_at	timestampz	No	
archived_at	timestampz	Yes	Soft-delete.

3. tasks

Column	Type	Null?	Notes
id	uuid	No	Primary key.
project_id	uuid	No	FK → projects.id.
title	text	No	FR-ADM-02.
instructions	text	No	Shown to Collector (C-06) and Admin (A-05).
reference_examples	jsonb	Yes	Array of reference media URLs.
created_at	timestampz	No	

4. task_assignments

Column	Type	Null?	Notes
task_id	uuid	No	FK → tasks.id; composite PK with user_id.
user_id	uuid	No	FK → users.id (role='collector').
assigned_by	uuid	No	FK → users.id (role='admin') — FR-ADM-03.
assigned_at	timestampz	No	
removed_at	timestampz	Yes	FR-ADM-04 — reassignment/removal, kept for audit rather than hard-deleted.

5. sessions

Column	Type	Null?	Notes
id	uuid	No	Primary key.
task_id	uuid	No	FK → tasks.id.
collector_id	uuid	No	FK → users.id.

Column	Type	Null?	Notes
started_at	timestamptz	No	
status	text	No	'in_progress' 'complete' — FR-SES-02; transition gated by stored procedure (Chapter 4.2).
notes	text	Yes	Collector-authored, Phase 2 (FR-SEC-01).

6. chunks

Column	Type	Null?	Notes
id	uuid	No	Primary key.
session_id	uuid	No	FK → sessions.id.
sequence_index	integer	No	Order within the session.
s3_object_key	text	No	UNIQUE — deterministic key, Volume 5.14.
status	text	No	'queued' 'uploading' 'failed' 'complete' (Chapter 2.8 status palette).
checksum_sha256	text	No	Set at capture time (FR-META-10).
file_size_bytes	bigint	No	
local_deleted_at	timestamptz	Yes	BR-08 — only set once status='complete'.

7. chunk_metadata

Column	Type	Null?	Notes
chunk_id	uuid	No	PK and FK → chunks.id (1:1, Chapter 4.3).
captured_start_at / captured_end_at	timestamptz	No	FR-META-02.
resolution / frame_rate / bitrate / codec / zoom_factor	text / int / int / text / numeric	No	FR-META-03.
device_model / os_version / app_version	text	No	FR-META-04.
gps_lat / gps_lng / battery_pct / network_type	numeric / numeric / int / text	Yes	FR-META-05 — nullable where permission/condition dependent.
notes_tags	jsonb	Yes	The one Collector-editable field (FR-META-07, BR-22).
verified_at	timestamptz	Yes	Set once backend confirms checksum match (FR-META-12).

8. audit_log

Column	Type	Null?	Notes
id	uuid	No	Primary key.
actor_id	uuid	No	FK → users.id.
action	text	No	e.g. 'project.create', 'collector.assign'.
target_table / target_id	text / uuid	No	
occurred_at	timestamptz	No	

9. What This Chapter Defers

Index definitions, exact CHECK constraint SQL, and migration ordering are Volume 6/7 implementation detail — this chapter fixes the schema's shape and meaning, not its migration scripts.

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.5

Metadata Specification

The Formal Schema Behind FR-META-01–15

Document Title	Metadata Specification
Chapter Reference	4.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Resolves	Volume 1, Ch.1.1 open question — exact metadata storage mechanism.

1. Storage Mechanism — Resolved

Volume 1 left this open: "dedicated table vs. document store vs. S3 object tags." Chapter 4.2/4.4 already answered it — a dedicated `chunk_metadata` table in Aurora Postgres, one row per chunk, 1:1 (Chapter 4.3). This chapter specifies the JSON wire format the mobile app (Chapter 3.9, `MetadataRepository`) sends and the backend stores/returns.

2. Canonical Metadata JSON Shape

```
{
  "chunk_id": "uuid",
  "identity": {
    "session_id": "uuid", "project_id": "uuid",
    "task_id": "uuid", "collector_id": "uuid", "device_id": "string"
  },
  "timing": {
    "sequence_index": 3, "started_at": "ISO-8601", "ended_at": "ISO-8601",
    "duration_seconds": 600
  },
  "capture": {
    "resolution": "1920x1080", "frame_rate": 30, "bitrate_kbps": 8000,
    "codec": "h264", "zoom_factor": 0.6, "camera": "rear-wide"
  },
  "device_context": {
    "device_model": "string", "os_version": "string", "app_version": "string"
  },
  "capture_conditions": {
    "gps": { "lat": 0.0, "lng": 0.0 },
    "battery_pct": 82, "network_type": "wifi"
  },
  "integrity": {
    "file_size_bytes": 512000000, "checksum_sha256": "hex-string"
  },
  "collector_authored": { "notes": "string", "tags": ["string"] }
}
```

Every field group maps directly to FR-META-01–07; the field-group names above match the ones Volume 1, Chapter 1.3 already used, so there is no re-labeling between requirement and implementation.

3. Write Path & Verification (FR-META-08–12)

- 1. Mobile app generates this full object the moment a chunk is finalized (FR-META-08), before any network call.
- 2. It is persisted to Drift locally first (FR-META-09), then uploaded to POST `/v1/chunks/{id}/metadata` alongside or immediately after the chunk's S3 upload (FR-META-11).
- 3. The backend independently verifies `integrity.checksum_sha256` against the actual uploaded S3 object (via a Lambda triggered on S3 PUT, or a direct recomputation call) before

setting `chunk_metadata.verified_at` and allowing `chunks.status` → 'complete' (FR-META-12, BR-21).

4. Immutability & Retention (BR-22/23, FR-META-13/14)

Immutability: every field above except `collector_authored` is written once, at creation, by the mobile app's authenticated request, and rejected on any subsequent PATCH by the database trigger named in Chapter 4.2 — enforced server-side, matching NFR-META-03's "zero exceptions, enforced server-side" requirement exactly.

Retention: `chunk_metadata` rows are never deleted when a chunk's local file is cleaned up (Volume 5, Chapter 5.15) or even when a Project is archived — only a future, explicit data-retention-policy job (Volume 8.7) may purge them, keeping metadata queryable by Admins (FR-ADM-06, NFR-META-04) for the full retention window.

5. Read Path — Admin Metadata Detail (A-08)

GET `/v1/chunks/{id}/metadata` returns the same JSON shape above plus `verified_at` and the chunk's current status, feeding Chapter 2.7's A-08 screen spec directly — the checksum is always returned paired with its verified/mismatch status, never as a bare value (Chapter 2.10, color/label-not-alone rule, applied here to data, not just UI).

6. What This Chapter Defers

- The export file format behind A-08's "Export Metadata" action — a follow-on decision for Volume 4's implementation phase, not fixed here since Chapter 3.7 already flagged it as open.
- Exact S3 checksum-verification trigger mechanics — Chapter 4.10 (Storage Architecture).

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.6

API Specification

The REST Endpoint Catalog

Document Title	API Specification
Chapter Reference	4.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Style	REST, versioned (/v1), resource-oriented, kebab-case — ADR-009, Chapter 4.1

1. Conventions

- Base path: /v1/... — a breaking change bumps to /v2 rather than mutating existing contracts.
- Auth: every request carries Authorization: Bearer <Firebase ID token> (Chapter 4.7); role/scope enforcement per Chapter 4.8.
- Standard envelope: { "data": ..., "error": null } on success, { "data": null, "error": { "code", "message" } } on failure — errors always carry a specific code, never a bare HTTP status alone, mirroring Chapter 2.9's named-cause-and-fix rule at the API layer.
- Pagination: cursor-based (?cursor=...&limit=...) on every list endpoint.

2. Auth & Users

Method	Endpoint	Role	Purpose
POST	/v1/auth/verify	Shared	Exchanges a Firebase ID token for the app's session context (role, org_id).
GET	/v1/users/me	Shared	Current user's profile + role.

3. Projects & Tasks

Method	Endpoint	Role	Purpose
GET	/v1/projects	Admin, Collector	Admin: all Projects in their org. Collector: only Projects with an assigned Task (BR-19).
POST	/v1/projects	Admin	FR-ADM-01.
GET	/v1/projects/{id}/tasks	Admin, Collector	Constitution's own naming example, verbatim.
POST	/v1/projects/{id}/tasks	Admin	FR-ADM-02.
PATCH	/v1/tasks/{id}	Admin	Edit instructions/reference examples.
POST	/v1/tasks/{id}/assignments	Admin	FR-ADM-03 — assign a Collector.
DELETE	/v1/tasks/{id}/assignments/{userId}	Admin	FR-ADM-04 — remove/reassign.

4. Sessions & Chunks

Method	Endpoint	Role	Purpose
POST	/v1/tasks/{id}/sessions	Collector	Starts a new recording session.
POST	/v1/sessions/{id}/chunks	Collector	Registers a finalized chunk; returns a presigned S3 multipart upload URL set.
PATCH	/v1/chunks/{id}/status	Collector (system-triggered)	Transitions queued → uploading → complete/failed; 'complete' gated server-side per Chapter 4.2.
POST	/v1/chunks/{id}/metadata	Collector	FR-META-11 — the JSON shape from Chapter 4.5.
GET	/v1/chunks/{id}/metadata	Admin	A-08 — FR-ADM-06.
GET	/v1/tasks/{id}/sessions	Admin	A-07 — session/chunk status across a Task.

5. Sample Request/Response — Chunk Registration

POST /v1/sessions/{id}/chunks

```
{ "sequence_index": 3, "file_size_bytes": 512000000,
  "checksum_sha256": "..."}

→ 201
{ "data": { "chunk_id": "uuid", "s3_object_key": "...",
            "upload_urls": ["presigned-part-1", "..."] },
  "error": null }
```

6. What This Chapter Defers

- Full OpenAPI/Swagger schema with every field type — an implementation artifact generated in Volume 6, not hand-maintained in this document.
- Rate limiting and quota policy — Volume 8 (Security & Compliance).

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.7

Authentication Design

Verifying Who Someone Is, Not What They're Allowed to Do

Document Title	Authentication Design
Chapter Reference	4.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Provider	Firebase Authentication — confirmed, Chapter 4.1

1. Flow

- 1. Collector/Admin signs in on the single shared Login screen (SH-02, Chapter 2.4 §4) via Firebase Authentication (email/password for MVP; SSO entry point reserved in the UI per Chapter 2.7).
- 2. The mobile app receives a Firebase ID token (a signed JWT) and attaches it as a Bearer token on every backend request (Chapter 4.6).
- 3. Each Lambda function verifies the token using the Firebase Admin SDK before touching any data — signature, expiry, and issuer are all checked; an invalid or expired token is rejected with a 401 before any database query runs.
- 4. On first verification, the backend looks up (or creates, on first login) the matching users row via `firebase_uid` (Chapter 4.4) and attaches `role + org_id` to the request context for Chapter 4.8's authorization layer.

2. Custom Claims for Role

Role (admin/collector) is set as a Firebase custom claim at account provisioning time (when an Admin invites a Collector, or when an org is first set up), not read from a request body — a client can never claim its own role. The claim is embedded in the ID token itself, so the backend can check it without an extra database round-trip on every request, falling back to the users table as the authoritative source if the claim and the table ever disagree.

3. Token Refresh & Session Expiry

Refresh: the Firebase client SDK on-device refreshes the ID token automatically before its ~1-hour expiry, transparent to the mobile app's AuthState (Chapter 3.9) as long as the refresh token itself is valid.

Expiry mid-upload: if a token expires while a background upload is in flight (Chapter 3.2, ADR-003), the upload's Dio auth interceptor (ADR-004) requests a fresh token and retries the request rather than failing the chunk outright — consistent with Chapter 2.9's rule that a transient condition should self-recover, not surface as a Collector-facing error.

Session expiry while genuinely signed out: routes the app to SH-02 (Login) via the Role Router, per Chapter 2.4 §4.

4. Verification Pseudocode

```
async function authenticate(request):
  token = request.header('Authorization').replace('Bearer ')
  decoded = firebaseAdmin.verifyIdToken(token) # throws on invalid/expired
  user = db.users.findByFirebaseUid(decoded.uid)
  if user is null: reject(401)
```

```
request.context.user = user    # role, org_id now available
return next()
```

5. What This Chapter Defers

- What each role/org context is then allowed to do — Chapter 4.8 (Authorization Design).
- Secret/key management for the Firebase Admin SDK service account — Volume 8 (Security & Compliance).

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.8

Authorization Design

Enforcing Volume 1's Roles and Business Rules on the Server, Not the Client

Document Title	Authorization Design
Chapter Reference	4.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Principle

Chapter 3.3's system architecture already stated it: the Backend API is the sole arbiter of what an authenticated user may read or write. No authorization decision is ever trusted from the mobile client — every endpoint in Chapter 4.6 re-derives role and scope from the verified token context (Chapter 4.7), on every request.

2. Authorization Middleware Chain

- 1. Authenticate (Chapter 4.7) — attaches user, role, org_id.
- 2. Role check — does this endpoint even permit this role? (e.g. POST /v1/projects requires role='admin', per FR-ADM-07.)
- 3. Scope check — is the specific resource (Project/Task/Session/Chunk) inside this user's allowed scope?
- 4. Only after all three pass does the resource handler run.

3. Scope Rules by Role

Admin: scoped to org_id (BR-20) — every Project/Task/Assignment/Metadata query is filtered WHERE org_id = :admin_org_id; an Admin can never read or write another org's data, regardless of guessed IDs.

Collector: scoped to their own task_assignments rows (BR-19) — every Project/Task/Session query is filtered to Tasks the Collector is currently assigned to; a removed assignment (Chapter 4.4, task_assignments.removed_at) immediately excludes that Task from all future queries, even if the mobile app's local cache hasn't refreshed yet.

4. Endpoint Permission Matrix

Action	Admin	Collector
Create/edit Project or Task	Allowed	Denied (FR-ADM-07)
Assign/remove Collector	Allowed	Denied
View own assigned Tasks/Sessions	N/A (sees all in org)	Allowed, own only (BR-19)
View Session/Chunk status across org	Allowed (FR-ADM-05)	Denied
View/export chunk metadata	Allowed (FR-ADM-06)	Denied — Collector never reads back metadata, only writes it at capture time
Create session / upload chunk	Denied (not their role)	Allowed, own assigned Task only
Edit chunk_metadata system fields	Denied	Denied — no role may edit them (BR-22)

5. Enforcement Is Structural, Not Optional

Because the scope filter (Section 3) is injected by shared middleware rather than repeated per-endpoint, a new endpoint added later cannot accidentally skip it — this is the same reasoning Chapter 3.7's coding standards apply to layer boundaries, applied here to authorization boundaries.

6. What This Chapter Defers

- The future Super Admin role's cross-org scope (Volume 1, Chapter 1.6) — deliberately out of MVP scope, revisited when Phase 3 begins.
- Full audit/compliance logging requirements — Volume 8 (Security & Compliance).

VOLUME 4 — BACKEND ARCHITECTURE

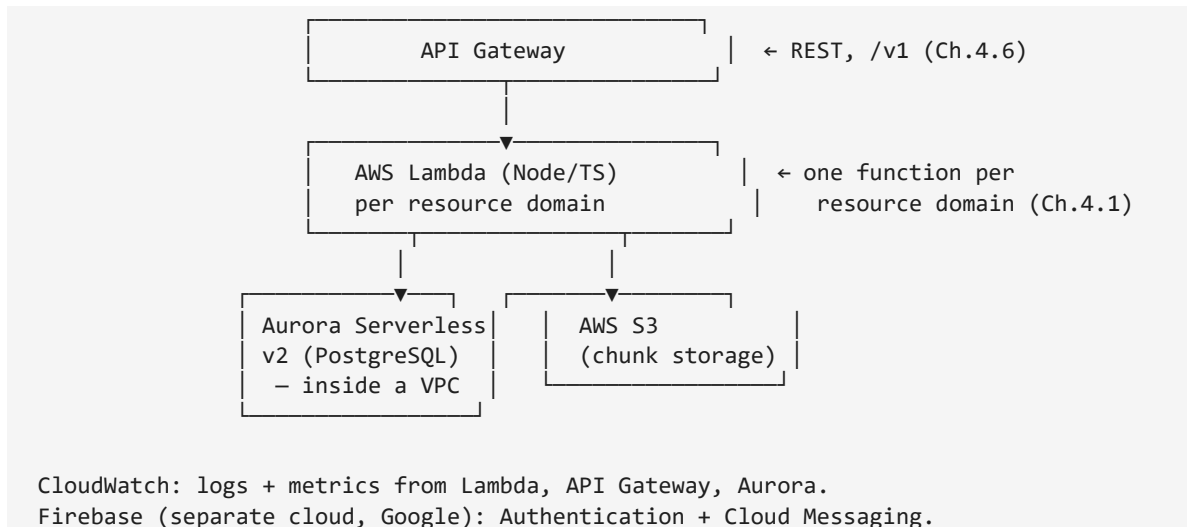
CHAPTER 4.9

Cloud Architecture

The AWS Account Topology Everything in Chapters 4.1–4.8 Runs On

Document Title	Cloud Architecture
Chapter Reference	4.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Topology (Textual Diagram)



2. Network & Security Boundaries

- Aurora Serverless v2 runs inside a private VPC subnet, reachable only from the Lambda functions (via a VPC-attached execution role) — never exposed to the public internet directly.
- API Gateway is the only public entry point; every Lambda function behind it is otherwise unreachable from outside AWS.
- IAM roles are least-privilege and per-function — e.g. the chunk-registration Lambda can generate presigned S3 URLs but cannot itself read arbitrary S3 objects; the metadata Lambda can write to chunk_metadata but has no S3 permissions at all.

3. Multi-Cloud Boundary (AWS + Firebase)

Why two clouds: Chapter 4.1's ADRs kept AWS for compute/storage/database (colocated, and S3 was already fixed by Volume 0) while confirming Firebase for Authentication/FCM, since Volume 0 already named it as the leading candidate and re-litigating it would contradict an already-reasonable prior decision.

The seam: the only integration point between the two clouds is Chapter 4.7's token verification — the AWS-side Lambda functions call the Firebase Admin SDK to verify a token, and nothing else crosses the boundary. This keeps the seam narrow and easy to reason about.

4. Environments

- Separate AWS accounts (or at minimum, separate VPCs/Aurora clusters and S3 buckets) for dev, staging, and production, following standard least-blast-radius practice — exact account structure finalized in Volume 7 (Development Environment).

5. What This Chapter Defers

- Infrastructure-as-code tooling (CDK/Terraform/CloudFormation) choice — Volume 7.
- Detailed IAM policy documents — Volume 8 (Security & Compliance).
- S3 bucket/prefix layout and lifecycle rules — Chapter 4.10, immediately following.

VOLUME 4 — BACKEND ARCHITECTURE

CHAPTER 4.10

Storage Architecture

How S3 Actually Holds and Protects Every Chunk

Document Title	Storage Architecture
Chapter Reference	4.10
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Bucket Layout

One S3 bucket per environment (dev/staging/prod, Chapter 4.9, Section 4); objects are organized by the deterministic key pattern finalized in Volume 5, Chapter 5.14 (the mobile-side File Naming Strategy), which this chapter treats as a fixed input rather than re-deriving it.

```
s3://human-archive-{env}/
  {org_id}/{project_id}/{task_id}/{session_id}/
  {sequence_index:04d}_{chunk_id}.mp4
```

2. Multipart Upload Flow

- 1. Mobile app calls POST /v1/sessions/{id}/chunks (Chapter 4.6); the Lambda computes the deterministic key (Volume 5.14) and returns a set of presigned multipart upload URLs — the mobile app never holds a raw AWS credential (NFR-SEC-03 stays satisfied even at the network boundary).
- 2. The Android foreground service / iOS background transfer (Volume 3, ADR-003) uploads each part directly to S3 using those presigned URLs.
- 3. On completion, the mobile app calls PATCH /v1/chunks/{id}/status; a Lambda (triggered either by that call or an S3 event notification) verifies the object exists and its size/checksum match the registered chunk (FR-META-12) before allowing 'complete'.
- 4. A retried/resumed upload reuses the same deterministic key and the same in-progress multipart upload ID where possible, which combined with the key's uniqueness constraint (Chapter 4.4) is what actually satisfies BR-11's "never a duplicate object" guarantee — the uniqueness is structural, not just a hope.

3. Encryption & Access

At rest: SSE-S3 (server-side encryption) enabled by default on the bucket; upgradeable to SSE-KMS with a customer-managed key if Volume 8's security review requires it.

In transit: HTTPS-only bucket policy; presigned URLs are HTTPS and time-limited (short expiry, re-requested if an upload takes longer than the window).

Access: the bucket has no public access at all; only the specific Lambda execution roles named in Chapter 4.9 can generate presigned URLs or read objects directly (e.g. for a future thumbnail/QA pipeline).

4. Lifecycle & Cost Management

Storage Class	When	Why
S3 Standard	0–30 days after upload	Recently captured footage is the most likely to be

Storage Class	When	Why
		reviewed/exported by an Admin (FR-ADM-06).
S3 Standard-IA	30–180 days	Access frequency drops sharply once a session is old and Complete.
S3 Glacier Instant Retrieval (or deeper, per Volume 8.7 retention policy)	180+ days	Long-term retention at minimal cost, while the metadata row (Chapter 4.5) stays fully queryable regardless of which storage class the raw file sits in.

None of these transitions ever delete an object outright without Volume 8.7's explicit retention policy sign-off — this chapter fixes the mechanism, not the exact day-counts, which are a policy decision, not an architecture one.

5. What This Chapter Defers

- The exact deterministic key algorithm — Volume 5, Chapter 5.14 (owned there since it must be computed identically on both the mobile client, for local file naming, and the backend, for the S3 key).
- Exact retention day-counts and legal-hold handling — Volume 8.7.